DeFuzz: Invariant-Guided Agentic Testing of Compiler-Instrumented Defenses

Anonymous Author(s)
Paper #XXX — IEEE S&P Double-Blind Review

Abstract—Compiler-instrumented defenses, such as stack canaries and control-flow integrity, mitigate the exploitation of memory-safety bugs. Yet these defenses can fail silently while compilation and functional tests still pass. CVE-2023-4039, for example, allows an overflow in a dynamically allocated buffer to bypass the GCC stack protector on AArch64. Such failures expose an oracle gap: ordinary crashes and output disagreements do not reliably reveal whether a defense still enforces its security guarantees.

We present DeFuzz, an agentic system for discovering silent failures in compiler-instrumented defenses through security invariants (properties required for a defense to hold under its threat model). The task poses three challenges: deriving these invariants, constructing checkers and test seeds that exercise them, and accounting for instruction set architecture (ISA) semantics. DeFuzz extracts candidate invariants from compiler source, architectural specifications, and historical bug reports and patches, retaining their supporting evidence. Accepted invariants guide both executable checkers and agent-generated seeds. Agents use source context, coverage measurements, and checker feedback to refine seeds toward potential violations, while ISA-aware checkers evaluate binary or runtime evidence under the relevant architectural assumptions. A fixed orchestration pipeline records this evidence for replay and keeps model output separate from bug adjudication. On GCC and LLVM, DeFuzz discovered [TBD: N] silent compiler defense failures, of which [TBD: M] were confirmed upstream. These findings expose weaknesses that functional compiler testing alone does not detect.

I. Introduction

Memory-safety bugs remain a threat to C and C++ software even after auditing and testing. Compiler-instrumented defenses, such as stack canaries, Control-Flow Integrity (CFI), and Pointer Authentication (PAC), provide an additional layer of protection when such bugs are triggered. We refer to these mechanisms as compiler defenses and study whether their emitted code enforces the intended security guarantees (Figure 1).

Depending on the mechanism, instrumentation is introduced in middle-end representations such as GIMPLE or LLVM IR, or during target-specific lowering. The compiler inserts checks, constrains layouts, or emits metadata to enforce a particular security policy. For example, a stack canary is checked before a function returns to detect overwrites that cross the guard.

Defense implementations are themselves code and can contain errors. Their effectiveness depends on both the instrumentation logic and its compatibility with the target instruction set architecture (ISA) and application binary interface (ABI). Incorrect checks, misplaced guards, or inconsistent assumptions across compilation stages can weaken a defense. The central question is whether the emitted implementation preserves its security guarantees under the semantics and conventions of the target architecture.

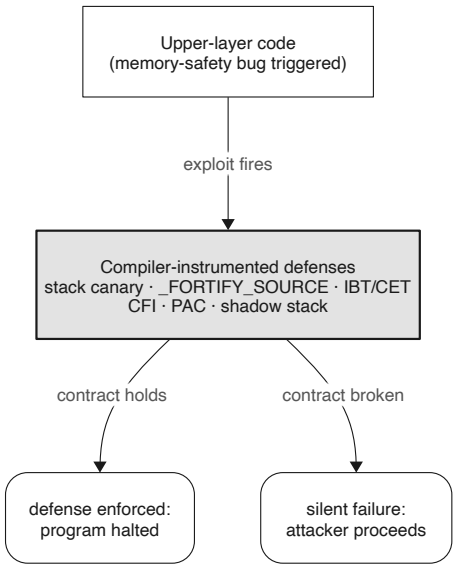


Fig. 1. Compiler-instrumented defenses mitigate attacks after an application-level memory-safety bug is triggered.

A. Silent Defense Failures

A compiler defense fails silently when compilation and ordinary functional tests succeed despite a violation of its security guarantees. CVE-2023-4039 exemplifies this problem: an overflow in a variable-length array (VLA) or alloca buffer could overwrite a saved return address without triggering the GCC stack protector on AArch64 [1]. The program still behaves correctly on non-attacking inputs, concealing the missing protection. We examine the faulty layout in §II.

Csmith [2] and YARPGen [3] use differential testing to expose compiler errors that change program behavior. Such output comparisons do not establish whether a defense remains effective: correct functional outputs can coexist with broken protection. This is the oracle gap for compiler defense testing.

B. Three Coupled Challenges

Our starting point is a security invariant: a property that a defense must satisfy under its threat model. Testing such properties can expose missing protection without requiring a functional miscompilation. Turning this idea into a systematic testing method presents three coupled challenges.

Deriving Security Invariants. Defense requirements are scattered across compiler documentation, source comments and assertions, ISA and ABI specifications, and historical fixes. They are rarely available as executable properties. Deriving an invariant requires identifying the protected asset, activation conditions, and required guarantee, while distinguishing security intent from implementation details that may themselves be faulty.

Constructing Invariant-Directed Tests. An invariant needs both a checker that recognizes a violation and a seed that exercises the relevant conditions. Generic program generation does not explicitly target this combination. For the stack protector, a seed must preserve the allocation and control-flow structure needed to expose a faulty guard placement, and the checker must observe that placement or its effect.

Accounting for ISA Semantics. The same security requirement can depend on different instruction semantics, register conventions, and stack layouts. Checkers must encode these assumptions and distinguish violations from valid target-specific implementations. The defense mechanism $\times$ ISA matrix therefore expands both the search space and the semantic work needed to interpret each result.

C. Our Approach

DeFuzz addresses these challenges with evidence-grounded invariant extraction, invariant-directed agentic testing, and ISA-aware checkers. Offline, it extracts candidate properties from compiler source, specifications, and historical bug reports and patches. Confirmed root causes also serve as retrieval probes for related implementations. Each accepted invariant retains its source evidence and is specialized to a concrete defense; transferring a failure pattern does not make the resulting invariant universal across mechanisms.

Online, each invariant–checker pair defines a testing objective. Large Language Model (LLM) agents use source-search tools and the invariant context to construct seeds, then revise them using measured coverage and checker feedback. This motivates the agentic design: generating a syntactically valid program is insufficient when the test must preserve specific semantic conditions, and one-shot generation cannot adapt to whether those conditions were reached. The agents perform this contextual refinement, while a fixed pipeline controls compilation and checking.

Static and dynamic checkers interpret evidence using target-specific instruction and ABI assumptions. Their metadata determines ISA applicability and routing, so unsupported cases are not mistaken for successful protection. Bug claims require deterministic binary or runtime evidence, not an LLM verdict. A versioned blackboard records seeds, feedback, and results for replay and ablation.

D. Contributions

In summary, this paper makes the following contributions:

- Evidence-Grounded Security Invariants. We derive defense-specific properties from documented requirements and historical failures, using root-cause retrieval to identify related candidates with explicit provenance (§IV, §V).
- Invariant-Directed Agentic Testing. We couple executable checkers with semantic seed generation and feedback, using explicit orchestration to make the search traceable and its components ablatable (§VII).
- ISA-Aware Oracles. We operationalize security invariants as static or dynamic checkers with explicit architectural applicability and deterministic evidence, addressing the oracle gap across supported targets (§VI).
- Empirical Discovery. On GCC and LLVM across multiple architectures, DeFuzz discovered [TBD: N] previously unknown silent compiler defense failures, of which [TBD: M] were confirmed upstream (§IX).

II. Background and Motivation

A. Compiler-Instrumented Defenses

A compiler-instrumented defense introduces checks, layout constraints, or metadata at compile time to enforce a security policy at run time. Enforcement may also require runtime libraries, operating-system support, or hardware features. Stack canaries use guard comparisons, CFI restricts indirect control-flow targets, and PAC uses signing and authentication instructions. The presence of such instrumentation alone does not establish that the intended protection holds.

Defense logic can span target-independent IR and backend representations such as RTL, SelectionDAG, or Machine IR. Lowering must respect instruction semantics and ABI conventions for registers, stack alignment, and call frames. For instance, dynamic allocations through alloca or VLAs require target-specific stack-pointer adjustments and can change the relation between buffers, saved registers, and guards. A property preserved by one backend may therefore fail in another. Figure 2 summarizes this mechanism $\times$ ISA space. Implementations may

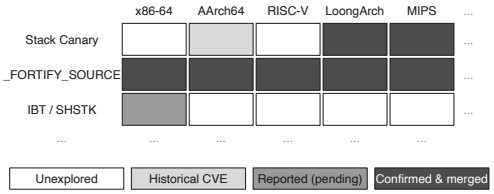


Fig. 2. The defense mechanism × ISA testing space: correctness on one target does not establish correctness on another.

share middle-end logic, but their target-specific behavior still requires separate validation.

B. Sources of Security Invariants

Security invariants express required protection, rather than merely describing behavior observed in an existing binary. Their evidence comes from three sources: compiler source and documentation, ISA and ABI specifications, and historical bug reports with their fixes. Documentation and specifications describe intended constraints; source comments and assertions identify where they apply; bug reports and patches expose assumptions that an implementation failed to preserve. Because the implementation under test can be wrong, its current behavior is not by itself a specification.

Recent work also separates property generation from verification. PropertyGPT [4] retrieves human-written smart-contract properties to generate new candidates and checks them with a dedicated prover. CIll [5] uses model-checking feedback to refine LLM-generated inductive invariants. These approaches motivate pairing generated properties with independent checking. Our task differs from proving a program assertion: we extract the security requirements of compiler defenses and search for compiled artifacts that violate them. An invariant must state its mechanism, applicability conditions, required property, and supporting evidence before it can guide a checker and a test seed.

C. Threat Model and Scope

We evaluate emitted defense code under the assumptions of each defense mechanism:

- Assumptions: The application contains a memory-safety vulnerability that an attacker can trigger. The defense is enabled and its required runtime and hardware support is present. Attacker capabilities are limited to the threat model of the mechanism being tested; a stack canary, for example, does not promise to stop arbitrary writes that skip the guard.

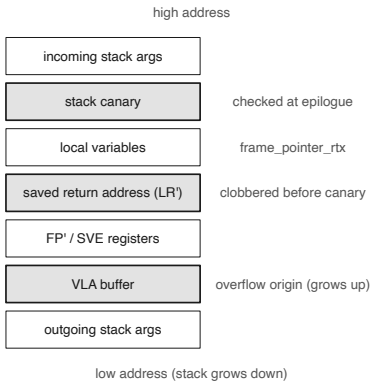


Fig. 3. CVE-2023-4039: an overflow from the VLA reaches the saved return address before the canary, leaving the guard unchanged.

- In-Scope: We study silent failures in compiler-instrumented defenses: compilation and ordinary functional tests succeed, but the emitted code violates a security invariant under the documented preconditions. Tests may inspect binary structure or exercise a violation at run time. They establish the failed protection without requiring an end-to-end exploit.
- Out-of-Scope: We do not search for application-level vulnerabilities or intentional source-level bypasses such as inline assembly that disables protection. Hardware vulnerabilities, side-channel attacks, and business logic flaws are also excluded. We evaluate standard defense emissions and exclude interactions between compilation optimization options and defense mechanisms.

The source corpus used for invariant extraction is pinned to GCC 16.1; bug-testing results are tied to the compiler version and target configuration recorded for each run.

D. Silent Failure: A Representative Case

CVE-2023-4039 illustrates the connection between a defense requirement, its architectural assumptions, and a testable violation [6]. For a contiguous stack overflow covered by the stack-protector policy, the guard must lie on the overwrite path to the saved return address. In the affected AArch64 layout, the saved return address lies at a higher address than the VLA or alloca buffer but below the canary (Figure 3). An overflow toward higher addresses can therefore corrupt the return address without changing the guard. The canary comparison still executes and passes.

The report traces this failure to a mismatch between target-independent guard placement and the AArch64 frame layout. It suggests both a seed containing a dynamic

allocation and a checker that resolves the relative locations of the allocation, guard, and saved return address. Merely finding a canary comparison in the binary would miss the failure.

A separate failure, GCC PR-96191 [7], concerns a canary value left in a temporary register after the AArch64 stack-protector check. The report cites the machine-description requirement that intermediate values must not survive the protection sequence. This yields a different invariant, about register state rather than stack layout, and requires a checker that follows the relevant register updates. The two cases show why the security property and its ISA-specific observation must both be explicit.

E. Why Existing Methods Miss It

Crash and differential testing can reveal functional miscompilations, but matching outputs on non-attacking inputs do not validate a defense. Moreover, attack-triggering executions may involve source-level undefined behavior, making ordinary cross-compiler output agreement unsuitable as the security oracle. Tests must instead check the protection promised by the enabled defense.

Static checking can detect a known violation when its rule models the relevant semantics. A symbol or instruction-presence rule, however, cannot establish correct guard placement or register cleanup. Extending such checks requires defense-specific properties and target-specific interpretation.

Generic seed generation does not directly connect those properties to the program structures that exercise them. Random generation may reach such structures, but coverage alone does not say whether a seed tests the assigned invariant. This motivates agents that inspect source context and revise seeds from checker feedback. Their role is to direct the search; executable checkers remain responsible for deciding whether the generated artifact violates the defense requirement.

III. System Overview

DeFuzz separates offline knowledge construction from online bug search. The offline stage reads compiler source, specifications, and historical bug records. A segmented CoT review provides broad candidate extraction, while historical-bug RAG targets implementations that may share a known root cause. Candidates that survive evidence grounding and deduplication enter the invariant catalog and are implemented as static or dynamic checkers. Confirmed findings can be added to the historical-bug pool as new retrieval probes.

The online stage evaluates the compiler's defense emission by orchestrating the compilation and subsequent execution of generated seeds. At run initialization, the orchestrator enumerates the checker catalog into a fixed queue. Each target denotes one invariant–checker pair; checker metadata supplies the applicable ISAs. The target remains fixed until its round or wall-clock budget is exhausted. Within each round, the Generator produces a C seed and a hard-wired pipeline executes routing $\rightarrow$ build $\rightarrow$ coverage $\rightarrow$ oracle. Pass, NotApplicable, and Error enter the Feedback branch before the next round; Error remains visible as an infrastructure diagnostic and consumes the current round without becoming a bug. A Fail invokes the Minimizer and is rechecked against the same checker. All intermediate states and results are versioned for replay and audit.

Figure 5 shows one target-specific iteration end to end; the following sections detail each stage.

IV. Security Invariants for Compiler Defenses

A. What is a security invariant

A security invariant is a property a defense must satisfy for it to be effective; violating it means the mechanism has been silently weakened in the compiled artifact or at run time, whether or not the code itself contains an overt bug. Each invariant is recorded with a machine-verifiable statement and, separately, an observation field that names only the externally observable phenomenon of a violation, not how to detect it. The separation of the phenomenon from its detection recipe is deliberate: "the ___stack_chk_fail symbol is missing from the binary" is a phenomenon, whereas "look it up in the ELF .dynsym" is a detection mechanism that belongs to the oracle implementation.

B. Corpus Construction and Candidate Extraction

The invariant archive is synthesized from three primary knowledge domains: compiler source code (specifically comments and runtime assertions), official ABI specifications, and historical vulnerability reports alongside their corresponding patches. To ensure interoperability with downstream analysis and continuous integration pipelines, every extracted invariant is formalized into a uniform schema. This schema explicitly captures the target mechanism, the originating source evidence, and the observable phenomena of a violation.

Given the vast scale of the compiler codebase, relevant semantic evidence is frequently fragmented across disparate compilation phases and backend templates. To address this, we partition the source material by mechanism and function boundaries. We then apply a segmented Chain-of-Thought (CoT) prompting strategy [8], instructing the model to isolate the protected asset, the activation conditions, and the required security property within each segment. It is critical to note that this pass strictly produces candidates. To prevent hallucination and ensure empirical validity, every candidate must retain its provenance and subsequently pass the stringent grounding and novelty gates detailed in §V.

C. Machine-Checkable Form: Static vs. Dynamic

Each invariant is classified by whether its violation can be observed in the un-executed binary (e.g., symbols,

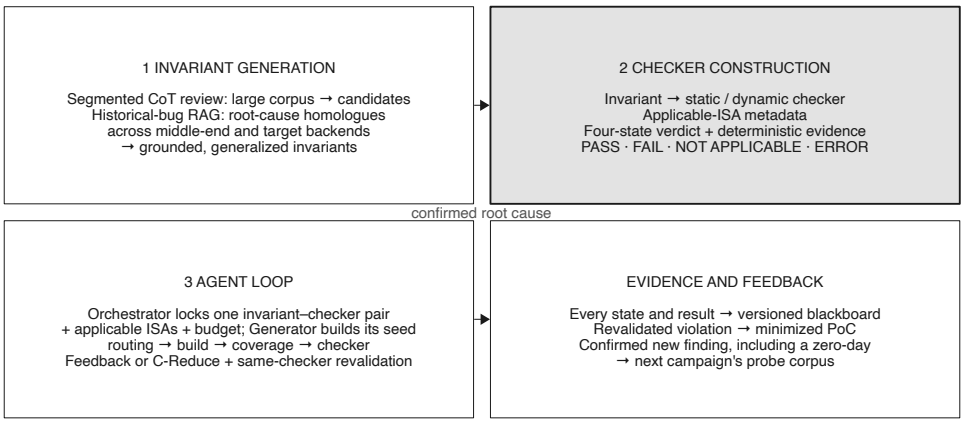


Fig. 4. DeFuzz technical route: corpus-grounded invariant generation feeds programmatic checkers, which define the targets of an explicitly orchestrated agent loop.

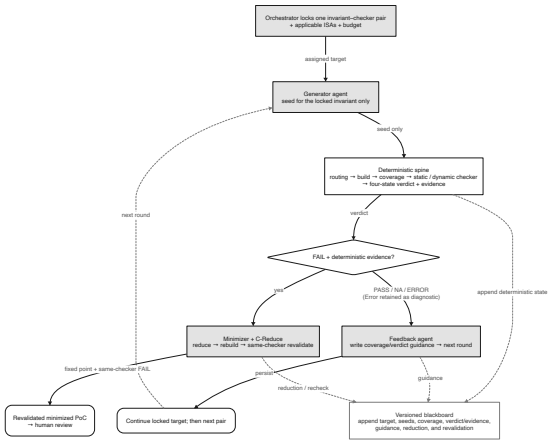


Fig. 5. Agent loop for one invariant–checker target and its applicable ISAs: generate → build → coverage → check → feedback or minimization.

sections, disassembly, strings) or if it strictly requires execution traces (e.g., exit status, output flags, register snapshots). This classification is guided by observability, decisiveness, computational cost, and attribution capability. To maintain precision, invariants exhibiting both static and dynamic violation signals are bifurcated into distinct checkers rather than being merged into a single heuristic.

V. Cross-Mechanism Invariant Transfer

A. Motivation: abstract-failure-mode transfer

Segmented review affords broad corpus coverage, yet it frequently fails to cluster isolated fragments that encode identical root causes, especially when the relevant evidence is fragmented across different mechanism names, compiler phases, or target backends. To bridge this gap, retrieval-augmented generation (RAG) [9] provides a complementary, high-yield path. By treating a confirmed historical bug as an explicit probe, the RAG phase actively searches for structurally related implementations. While this approach does not guarantee completeness, it significantly increases the probability of uncovering analogous defects that share a known root cause.

The RAG process begins by distilling a confirmed bug into a mechanism-agnostic failure signature. Retrieval then scans the indexed middle-end and backend corpora for implementations sharing similar semantic structures. We deliberately eschew entity-centric retrieval (e.g., matching API or symbol names), as it tends to confine results within the original defense mechanism. Instead, queries are formulated around abstract operations, violated assumptions, and protected assets, allowing a single probe to uncover analogous vulnerabilities across diverse mechanisms and backends.

B. Pipeline: Distillation, Analogy, Specialization, and Entailment

The generalization pipeline operates in four stages. First, a confirmed root cause undergoes distillation into

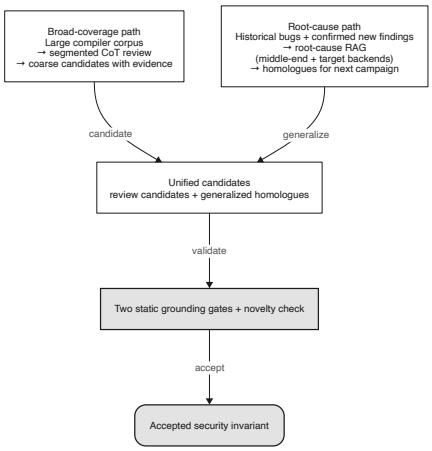


Fig. 6. Corpus-grounded invariant generation: segmented CoT review and historical-bug RAG feed a shared grounding and novelty filter; confirmed new findings return as retrieval probes.

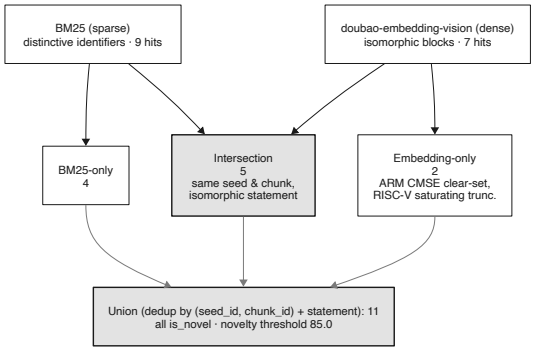


Fig. 7. Union/dedup of BM25 and embedding retrieval: 5 intersection, 4 BM25-only, 2 embedding-only, 11 union.

an abstract description. For instance, using CVE-2023-4039 as a running example, the stack canary vulnerability is distilled from "AArch64 VLA overrides canary" into the abstract semantic signature "dynamically sized local allocations bypass statically computed security boundaries." Second, an analogy filter evaluates whether a retrieved code block embodies the same structural failure, proactively rejecting superficial lexical matches. Third, specialization instantiates the invariant within the concrete target source context (e.g., mapping the abstract boundary to specific shadow-stack frame calculations). Finally, entailment verifies that the resulting statement logically follows from the cited evidence. Figure 6 illustrates how segmented review and the RAG path converge into a shared grounding and novelty filter.

C. Retrieval: BM25 and embedding are complementary

To effectively query the codebase, each distilled root cause is synthesized into a dual-aspect query structure: a root_cause_phrase that provides an abstract semantic summary for dense embeddings, and critical_tokens that capture mechanism-specific identifiers (e.g., register names, macro flags) for sparse retrieval.

Within this hybrid RAG stage, we evaluate sparse BM25 and dense doubao-embedding-vision retrieval over the same 24 probe seeds and the same 4,496 GCC-16.1 source chunks. BM25 excels on the critical_tokens (ix86_zero_call_used_regs, SUBREG_PROMOTED, CONST_HIGH_PART); the dense retriever utilizes the root_cause_phrase to pull in semantically isomorphic blocks whose lexical anchors are weak (the CMSE clear-set constructor, the RISC-V saturating truncation). Deduplicating by (seed_id, chunk_id) and statement, the two paths yield 5 in the intersection, 4 BM25-only, and 2 embedding-only, for 11 in the union (Figure 7); all 11 are marked is_novel after a novelty check (threshold 85.0). Table I summarizes.

TABLE I
Transferred invariants by retrieval backend (24 probes, 4,496 GCC-16.1 chunks).

Category	Count	Note
Intersection (BM25 ∩ embed)	5	same seed & chunk, isomorphic statement
BM25-only	4	embed missed the block for that seed
Embedding-only	2	BM25 missed; dense retrieval added
Union (deduplicated)	11	$9 + 7 - 5$; all is_novel

D. Grounding gates and reproducibility

Candidates from both segmented CoT review and historical-bug RAG enter the same acceptance path. An invariant is accepted only after two static grounding gates confirm the cited evidence supports its statement and observation, and after the novelty check (threshold 85.0) confirms it does not duplicate an existing seed or manual rule. Because the dense retriever returns slightly different vectors per call, we cache query vectors keyed by query text so that top-k ordering, and hence the accepted set, reproduces across runs. After a new finding, including a zero-day, is reproduced and its root cause is confirmed, we serialize it in the same schema and add it to the probe pool. The next campaign can then search for further instances of that root cause.

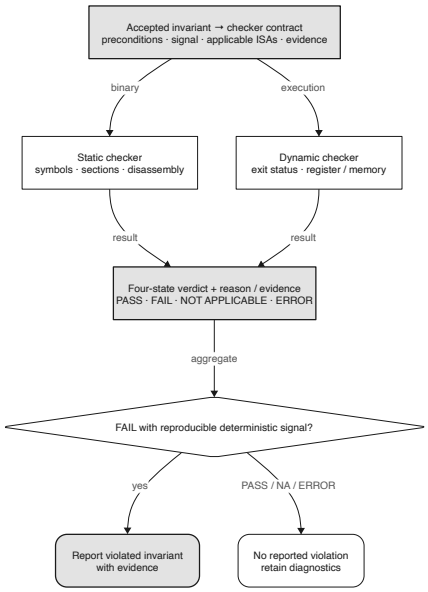


Fig. 8. An accepted invariant is translated into a static or dynamic checker; four-state verdicts pass through a deterministic-evidence gate before any finding is confirmed.

VI. Oracle: From Invariants to Verdicts

A. Checker design

To operationalize an accepted invariant into an executable oracle, we define a formal checker contract. Let $\mathcal{P}$ be the space of compiled programs, $\mathcal{I}$ be the set of target ISAs, and $\mathcal{S}$ be the set of accepted invariants. An oracle is defined as a deterministic function:

$$\mathcal{O} : \mathcal{P} \times \mathcal{I} \times \mathcal{S} \to \{\text{Pass}, \text{Fail}, \text{NA}, \text{Error}\} \tag{1}$$

Invariants decidable without execution are implemented as static checkers analyzing symbols, sections, or disassembly. Conversely, invariants requiring runtime context (e.g., exit status, memory states) are realized as dynamic checkers. Invariants amenable to both are implemented redundantly, preserving deterministic precision over heuristic approximation.

Each checker returns a four-state verdict (pass / fail / not-applicable / error). NotApplicable is returned when the mechanism is off or the seed does not exercise it, and is ignored by the aggregation layer. A finding is counted as confirmed only when a deterministic checker fails or an execution differential reproduces, never on the strength of model output alone (Figure 8).

When a programmatic checker cannot decide an invariant–ISA cell, it returns NotApplicable or Error. The LLM is strictly prohibited from inferring or overriding verdicts based on disassembly or symbol evidence; all Fails must derive from deterministic execution or static analysis rules. Consequently, no provisional aggregate is ever promoted to violated by model output alone, ensuring that minimization and final bug claims are strictly grounded in reproducible evidence.

B. Mechanism aggregator and flag profiles

A per-mechanism aggregator combines checker verdicts: any Fail raises a mechanism-level violation carrying its evidence. On/off flag profiles let the oracle adjudicate differentially (defense enabled vs. disabled), and because the checkers key on binary and runtime signals rather than on the producing compiler, GCC and LLVM share the same checkers.

C. Checker metadata as a single source of truth

Each checker declares three metadata fields: which ISAs it binds, whether it is single-ISA or differential, and whether it is cheap or expensive. This metadata is the single source of truth read by both the gRPC and the MCP adapter, and it is what makes the ISA routing in §VII a table lookup rather than an agent decision.

VII. Agentic Loop

A. Design principles

The orchestrator, rather than an agent, owns target selection and control flow. At run initialization it constructs a fixed queue of invariant–checker pairs. For each target, checker metadata determines the applicable ISAs, and the target remains fixed for a configured round or time budget. Within a round, the pipeline follows the same order: generate → routing → build → coverage → oracle → route. Agents are invoked only at fixed positions and communicate through shared state. The Generator proposes; the oracle stack adjudicates, and confirmed findings require deterministic evidence.

B. Explicit orchestration and the blackboard

The agents form a closed loop where feedback guides subsequent generation, the oracle adjudicates outcomes, non-violations update feedback parameters, and violations trigger minimization. Crucially, all inter-agent communication flows through a versioned blackboard rather than direct dialogue (Figure 9). Algorithm 1 details this deterministic control flow.

This architecture enforces reproducibility (as pinning a blackboard version deterministically fixes agent inputs), ablatability (enabling independent toggling of linkage edges), and auditability (ensuring every bug claim traces back to deterministic execution evidence).

C. Three agents

The Generator (invoked at the start) carries read-only source-search and invariant-query tools and outputs a seed for the assigned invariant–checker pair; coverage is not an agent-reportable tool but is measured by the coverage node and fed back, so the agent cannot fabricate coverage. The feedback agent (invoked on a non-violation) is a

Algorithm 1 Explicitly Orchestrated Agentic Loop

Require: Queue Q of invariant–checker targets, Budget B

Initialize Blackboard $\mathcal{B}$
for target $T \in Q$ do
  while budget $B > 0$ do
    $\mathcal{B}$.seed $\leftarrow$ Generator($\mathcal{B}$.feedback)
    $\mathcal{B}$.binary $\leftarrow$ Compile($\mathcal{B}$.seed, T.ISA)
    v, cov $\leftarrow \mathcal{O}(\mathcal{B}$.binary, T.ISA, T.inv)
    if $v ==$ Fail then
      $\mathcal{B}$.poc $\leftarrow$ Minimizer($\mathcal{B}$.seed, $\mathcal{O}$)
      yield BugFound($\mathcal{B}$.poc)
    else if $v ==$ Pass $\vee$ $v ==$ NA then
      $\mathcal{B}$.feedback $\leftarrow$ FeedbackAgent(cov, v)
    end if
    $B \leftarrow B - 1$
  end while
end for

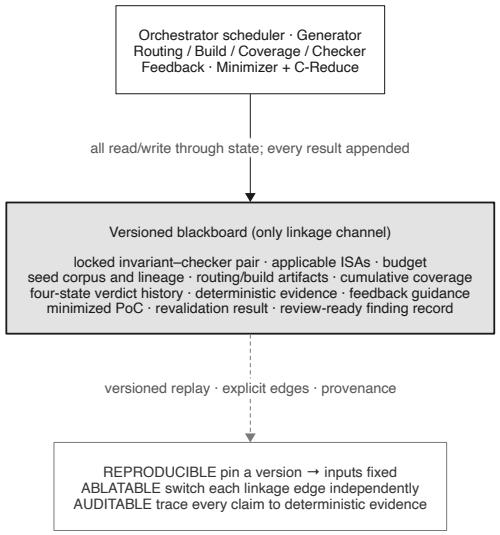


Fig. 9. The blackboard: all inter-agent linkage flows through versioned shared state, enabling reproduce / ablate / audit.

context-isolated subagent that turns the coverage delta, checker identity, and the Pass/NotApplicable verdict into next-round guidance written back to the blackboard. The minimizer (invoked on a violation) shrinks the PoC with deterministic delta debugging (C-Reduce) as the workhorse and the LLM only for semantic guidance, so that reduction does not accidentally delete the structure that triggers the bug. Each candidate reduction is compiled and rechecked with the same checker before it is retained.

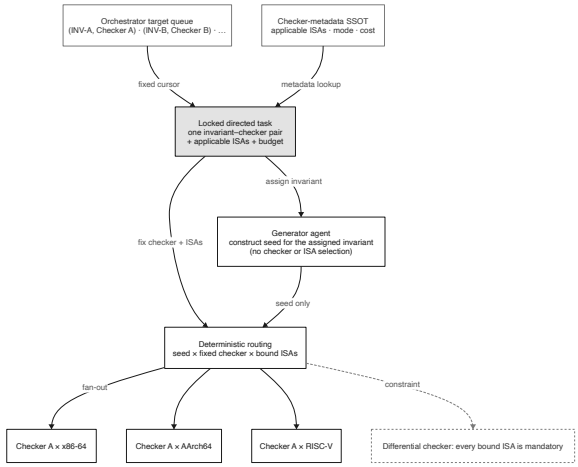


Fig. 10. Deterministic target scheduling: one invariant–checker pair remains fixed for its budget, while checker metadata expands the applicable ISAs and cheap static checkers remain enabled.

D. Checker-routed seeds, ISA-bound checkers

Each target in the deterministic queue denotes one invariant and its checker. The routing node expands that checker to every ISA in its metadata; the Generator never chooses either the target invariant or an ISA. A differential checker always expands to its full ISA set so that a cross-ISA signal cannot be pruned by model behavior. Cheap static checkers remain enabled as a side path, but they do not change the Generator's assigned target. When a checker returns Pass or NotApplicable, the Feedback agent records the relevant context and the same pair continues until its budget expires. Error follows the same non-bug path, remains visible as an infrastructure diagnostic, and consumes the current round. A Fail transfers the seed and deterministic evidence to the Minimizer. The blackboard records both branches before the orchestrator advances.

VIII. Implementation

The DeFuzz architecture is decoupled into a high-performance evaluation core and a Python-based agentic orchestrator. The core service, implemented in Go, provides deterministic build execution, coverage tracking, and oracle adjudication. It exposes these capabilities to the orchestrator via gRPC for high-throughput programmatic access, and via the Model Context Protocol (MCP) to provide agents with read-only tools for source code search, invariant querying, and delta debugging (C-Reduce) invocation. Crucially, the oracle definitions, static/dynamic checkers, and ISA applicability rules are centralized within the Go core, serving as the single source of truth for all metadata.

The orchestrator governs the explicit agentic loop, maintaining the strict topological order of generation,

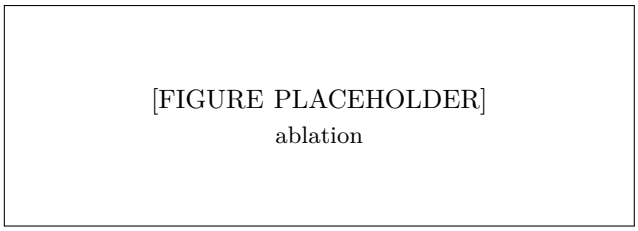


Fig. 11. Ablation: contribution of explicit orchestration (vs. free-running agents), oracle grounding, and coverage feedback to hit rate and cost.

routing, build, and oracle adjudication. It manages the versioned blackboard, which tracks the lineage of seed corpora, cumulative coverage metrics, and feedback guidance. To ensure rigorous auditability, every execution persists a dedicated artifact bundle containing a serialized state chain, full toolchain configurations, and LLM hyperparameters. This design enables deterministic replay and post-hoc attribution of any confirmed bug claim.

IX. Evaluation

Setup. [DEFERRED: Awaiting experimental data. Setup must specify LLM parameters (model, temperature), baseline configurations (free-running agent setup), and exact computational budget/hardware for reproducibility.]

a) RQ1: Real bugs: [DEFERRED: Awaiting experimental data. Needs confirmed bug counts and CVE IDs.]

b) RQ2: Invariant generation quality: [DEFERRED: Awaiting experimental data. Requires expert sampling grading (True Security Property, Reasonable but Non-essential, Trivial, Incorrect) and Cohen's κ calculation.]

c) RQ3: Ablation: [DEFERRED: Awaiting experimental data. Requires evaluation of explicit orchestration vs. free-running baseline, coverage feedback, and oracle grounding.]

d) RQ4: Cross-ISA generalization: [DEFERRED: Awaiting experimental data.]

X. Discussion and Limitations

[DEFERRED: Awaiting experimental data. Must include: (1) Manual effort analysis for converting invariants to checkers, (2) LLM API token cost/budget analysis, (3) False positive analysis, specifically regarding QEMU simulation fidelity vs. bare-metal hardware.]

XI. Related Work

Compiler Bug Finding and The Oracle Gap. Traditional compiler fuzzing has achieved immense success in uncovering optimization bugs and crashes. Tools like Csmith [2], YARPGen [3], and EMI [10] rely fundamentally on differential testing or crash detection as their oracle. However, these methods possess a structural blind spot regarding security defenses. A silent failure, by definition, preserves the functional semantics of the source code; the compiled artifact executes correctly and its output matches that of unoptimized or cross-compiler builds. Because there is no semantic divergence or runtime crash, differential and crash-based oracles are theoretically incapable of detecting these security contract violations.

Silent and Security Bugs in Compilers. The recognition of silent security failures in compilers is gaining traction. Studies such as "Silent Bugs Matter" [11] provide crucial empirical taxonomies of compiler-introduced security bugs, confirming that optimizations can inadvertently eliminate security checks or introduce timing side-channels. However, these works are primarily measurement studies and do not provide an automated, generative system for detecting new instances of such failures across different backends. DeFuzz bridges this gap by operationalizing these observations into an executable oracle.

LLM and RAG-based Property Generation. Recent advancements leverage Large Language Models to deduce system properties. PropertyGPT [4] employs Retrieval-Augmented Generation to synthesize smart-contract invariants, while SpecAuditor [12] identifies specification violations through semantic generalization. A critical distinction in DeFuzz is our retrieval strategy. Existing methods predominantly rely on entity-centric retrieval (e.g., matching specific API or variable names). In contrast, DeFuzz deliberately rejects entity-driven matching in favor of abstract-failure-mode transfer (§V), utilizing dual-aspect queries to bridge the lexical divide between disparate defense mechanisms (e.g., stack canaries vs. PAC) and architectures.

LLM-Agent Fuzzing: Hallucination vs. Determinism. The integration of LLMs as autonomous agents in fuzzing pipelines (e.g., AgentFuzz [13]) introduces powerful directed exploration capabilities. However, free-running agent architectures suffer from severe reproducibility and hallucination issues. When an LLM is granted the autonomy to decide whether a complex output constitutes a "bug," the system inevitably yields false positives driven by model hallucination. DeFuzz structurally diverges from this paradigm through explicit orchestration (§VII). By confining the LLM to generation and feedback roles, and strictly mandating that all bug claims be grounded in the deterministic execution of programmatic checkers, DeFuzz eliminates hallucination from the adjudication phase, ensuring that every reported violation is a reproducible, actionable defect.

XII. Conclusion

Compiler-inserted security defenses represent a critical backstop against memory-safety exploits, yet their efficacy is frequently compromised by silent failures during backend lowering. These failures, which bypass traditional crash and differential fuzzing oracles, leave binaries seemingly protected but functionally vulnerable across the mechanism $\times$ ISA matrix.

DeFuzz addresses this fundamental blind spot. By distilling defense semantics into a deterministic, machine-checkable oracle, we transition the detection of silent failures from ad-hoc manual auditing to a systematic, reproducible science. Our cross-mechanism invariant transfer pipeline demonstrates that historical root causes can be abstracted and transferred to uncover analogous flaws in disparate defenses. Furthermore, our explicitly orchestrated agentic loop proves that LLMs can be effectively harnessed for complex fuzzing tasks without succumbing to adjudication hallucinations, provided their outputs are strictly grounded in deterministic execution evidence.

Ultimately, DeFuzz discovered [DEFERRED: N] previously unknown silent failures in production compilers, resulting in [DEFERRED: M] CVEs. These findings underscore not only the fragility of current compiler defenses but also the necessity of integrating rigorous, invariant-grounded oracles into the compiler development lifecycle.

References

[1] MITRE, "CVE-2023-4039: GCC AArch64 stack protector bypass with variable-length arrays," https://nvd.nist.gov/vuln/detail/CVE-2023-4039, 2023.

[2] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2011.

[3] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for C and C++ compilers with YARPGen," in Proceedings of the ACM on Programming Languages (OOPSLA), 2020.

[4] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2025.

[5] Y. Su, T. Bu, Q. Yang, Y. Ci, and E. Tian, "CIll: CTI-guided invariant generation via LLMs for model checking," arXiv preprint arXiv:2602.23389, 2026. [Online]. Available: https://arxiv.org/abs/2602.23389

[6] T. Hebb, "CVE-2023-4039: GCC's -fstack-protector fails to guard dynamic stack allocations on ARM64," https://rtx.meta.security/mitigation/2023/09/12/CVE-2023-4039.html, 2023.

[7] GCC Bugzilla, "Bug 96191: aarch64 stack_protect_test canary leak," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=96191, 2020.

[8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in Advances in Neural Information Processing Systems, vol. 35, 2022, pp. 24 824–24 837.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

[10] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2014.

[11] G. A. Di Luna et al., "Silent bugs matter: A study of compiler-introduced security bugs," in Proceedings of the USENIX Security Symposium, 2023.

[12] M. Lin and H. Chen, "SpecAuditor: Detecting specification violations via semantic generalization," in IEEE Symposium on Security and Privacy (S&P), 2026.

[13] F. Liu et al., "Testing the robustness of LLM-based agents via directed greybox fuzzing," in Proceedings of the USENIX Security Symposium, 2025.